

word_tokenize/EN hangs on incorrect strings #2866

Closed

#2869

raffienficiaud opened on Oct 27, 2021 · edited by raffienficiaud

Edits ···

Hi NLTK team,

I have a string that I pass to `word_tokenize` (which uses `PunktSentenceTokenizer` under the hood), and the call hangs. The string in question is taken from Wikipedia and is the result of some vandalism. It can be generated by this

```
text = 'swirley thing w' + 'e' * (884779 - len('swirley thing w'))
```



The call seems to hang, I did not go deep too much but after running this for a couple of hours I just stopped the process.

What would be an ok solution to process this in a robust fashion? I have a pipeline that has correct sentences as well as, from time to time, this kind of sentences.

tomaarsen on Oct 27, 2021 · edited by tomaarsen

Edits ··· Member

Hello [@raffienficiaud](#)!

You seem to have stumbled upon a ReDOS, which is a serious (security) problem - as you have already noticed. I'll be attempting to chase it down. However, that clearly isn't much help to you in the moment.

For the time being, you can apply some pre-processing to the text. In particular, you may check whether the length of the input text is realistic. Beyond that, some other steps can be taken too, like verifying whether the input text is realistic based on the number of spaces.

That said, I don't want to saddle you with a bunch of work for pre-processing.

Another solution from our end would be to allow users to supply a timeout to our Regex-based tokenizers, so developers can be sure of an upper bound of the execution time - although they would then have to deal with the "failed" cases. I'd rather make sure our tokenizers aren't weak to any ReDOS vulnerabilities, but this would be a nice insurance that users won't have their programs hang for hours after receiving malicious input.

Preliminary tests show that an input string that is twice as long has a roughly 3-4 times execution time. This is far from ideal considering we're still only dealing with 3 words here.

A length of 512	takes 0.0020s
A length of 1024	takes 0.0061s
A length of 2048	takes 0.0230s
A length of 4096	takes 0.0920s
A length of 8192	takes 0.3610s
A length of 16384	takes 1.4300s
A length of 32768	takes 6.1325s
A length of 65536	takes 26.0703s



 **tomaarsen** added **bug** **critical** **tokenizer** on Oct 27, 2021



 **tomaarsen** mentioned this on Oct 27, 2021

 Resolved serious ReDoS in PunktSentenceTokenizer #2869

 tomaarsen on Oct 28, 2021

Member ...

If this issue is critical to you, then you can install NLTK from the `tomaarsen:bugfix/punkt-tokenizer-redos` branch used in [#2869](#), where I try to resolve this ReDoS. Otherwise, I presume that the next release will have fixed this issue.

Thanks a lot for bringing this to our attention!

 raffienficiaud on Oct 28, 2021

Author ...

Hi [@tomaarsen](#)

Thanks for all those suggestions, your investigation and the very quick fix (and overall improvement). Currently I am just discarding the document that causes the issue, and I would be more prone in upgrading NLTK version when the fix is ready/merged. There is nothing urgent on my side, my processing is mostly offline.

(I let you close the issue when you think it is the most convenient.)

Thanks again for your time!

 tomaarsen on Oct 28, 2021

Member ...

[@raffienficiaud](#)

Gladly, happened to have some spare time today, and it was an interesting problem to tackle!

The issue has been linked to the PR, so it will auto close if the PR gets merged.

- Tom Aarsen



tomaarsen self-assigned this on Nov 5, 2021



stevenbird closed this as completed in [#2869](#) on Nov 26, 2021

Sign up for free

to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

Assignees



tomaarsen

Labels

bug

critical

tokenizer

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development



Code with agent mode



Resolved serious ReDoS in PunktSentenceTokenizer

nltk/nltk

Participants



